

How to Store Fixed and Dynamic Values in Destination Property in AutoMapper

In this article, I am going to discuss How to Store Fixed and Dynamic Values in Destination Property in AutoMapper with Examples using C# Language. Please read our previous article where we discussed the [AutoMapper Ignore Method in C#](#) with Examples.

How to Store Fixed and Dynamic Values in Destination Property in AutoMapper

The AutoMapper UseValue() method is used to assign a Fixed Value to a Destination Property whereas the ResolveUsing() is used to assign a dynamic value to a destination property at runtime. But, from AutoMapper 8.0, UseValue() and ResolveUsing() methods are deprecated and if you are trying to use the above two methods, then you will get compile time error. So, whatever we achieve using UseValue() and ResolveUsing() methods, now we can achieve the same using the AutoMapper MapFrom() method.

Let us understand this with an example. To understand how to assign Fixed and Dynamic Values to Destination Properties using AutoMapper, we are going to use the following two classes. Create a class file with the name PermanentAddress.cs and then copy and paste the following code into it. This is a simple class having four properties.

```
using System;
namespace AutoMapperDemo
{
    public class PermanentAddress
    {
        public string Name { get; set; }
        public string Address { get; set; }
        public string CreatedBy { get; set; }
        public DateTime? CreatedOn { get; set; }
    }
}
```

Next, create another class file with the name TemporaryAddress.cs and then copy and paste the following code into it. This is also a very simple class having four properties.

```
using System;
namespace AutoMapperDemo
{
    public class TemporaryAddress
    {
        public string Name { get; set; }
        public string Address { get; set; }
        public string CreatedBy { get; set; }
        public DateTime? CreatedOn { get; set; }
    }
}
```

Business Requirement:

Our business Requirement is to Map to Source PermanentAddress Object to the Destination TemporaryAddress object. But, we don't want to map the CreatedBy and CreatedOn property values of the Source Object to the Destination Object. Instead, we want to store the hard-coded or Fixed value "System" as the Value for the CreatedBy column and the current DateTime value as the Value from the CreatedOn destination Property. Before AutoMapper 8.0, we use UseValue and ResolveUsing methods. But, now we need to use the MapFrom method to assign fixed and dynamic values to the destination property.

So, create a class file with the name MapperConfig.cs and then copy and paste the following code into it. As you can see, the following class contains two methods i.e. InitializeAutomapper and GetCurrentDateTime. Within the InitializeAutomapper method, we are providing the mapping configuration, and the GetCurrentDateTime method is used to return the current system data time. Further, you can notice, using the ForMember and MapFrom methods we are assigning the hard-coded and fixed value System to the CreatedBy destination Property and assign the dynamic value from the destination CreatedOn property, within the MapFrom method, we are calling the GetCurrentDateTime method and whatever dynamic value GetCurrentDateTime method returns that we are assigning to the CreatedOn method.

```
using AutoMapper;
using System;
namespace AutoMapperDemo
{
    public class MapperConfig
    {
        public static Mapper InitializeAutomapper()
        {
            //Configuring AutoMapper
            var config = new MapperConfiguration(cfg =>
            {
                cfg.CreateMap<PermanentAddress, TemporaryAddress>()
                    //Using MapFrom Method to Store Static or Hard-Coded Value
                    //Destination Property
                    .ForMember(dest => dest.CreatedBy, act => act.MapFrom(src =>
                    "System"))
                    //Before AutoMapper 8.0, to Store Static Value use the UseValue
                    //method
                    //.ForMember(dest => dest.CreatedBy, act => act.UseValue('System'))
                    //Using MapFrom Method to Store Dynamic Value in a Destination
                    //Property
                    //Here, we are calling GetCurrentDateTime method which will return
                    //dynamic value
                    .ForMember(dest => dest.CreatedOn, opt => opt.MapFrom(src =>
                    GetCurrentDateTime()))
                    //Before AutoMapper 8.0, To Store Dynamic value use ResolveUsing
                    //method
                    //.ForMember(dest => dest.CreatedBy, act => act.ResolveUsing(() =>
                    //{
                    //    return DateTime.Now;
                    //}))
                    .ReverseMap();
            });

            //Creating the Mapper Instance
            var mapper = new Mapper(config);
        }
    }
}
```

```

        //returning the Mapper Instance
        return mapper;
    }
    //Method to return Dynamic Value
    public static DateTime GetCurrentDateTime()
    {
        //Write the Logic to Get Dynamic value
        return DateTime.Now;
    }
}
}

```

Now, to check whether the Fixed Value and Dynamic Value working as Expected, modify the Main Method of the Program class as follows. Here, we are not storing the CreatedOn and CreatedBy values in the Source Object, but while you run the application, then you will see the Fixed and Dynamic Values which we set using AutoMapper for the CreatedOn and CreatedBy properties of the destination object.

```

using System;
namespace AutoMapperDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Initialize AutoMapper
            var mapper = MapperConfig.InitializeAutomapper();

            PermanentAddress permAddress = new PermanentAddress()
            {
                Name = "Pranaya",
                Address = "Mumbai"
            };
            var TempAddress = mapper.Map<PermanentAddress, TemporaryAddress>
(permAddress);
            Console.WriteLine("After Mapping Permanent Address");
            //Here CreatedBy and CreatedOn will be empty for Permanent Address
            Console.WriteLine($"Name: {permAddress.Name}, Address: {permAddress.Address},
CreatedBy: {permAddress.CreatedBy}, CreatedOn: {permAddress.CreatedOn}");
            Console.WriteLine("After Mapping Permanent Address");
            //Here CreatedBy with Fixed Value and CreatedOn with Dynamic Value
            Console.WriteLine($"Name: {TempAddress.Name}, Address: {TempAddress.Address},
CreatedBy: {TempAddress.CreatedBy}, CreatedOn: {TempAddress.CreatedOn}");
            Console.ReadLine();
        }
    }
}

```

When you run the application, it will give you the following output.

```
After Mapping Permanent Address
Name: Pranaya, Address: Mumbai, CreatedBy: , CreatedOn:
After Mapping Permanent Address
Name: Pranaya, Address: Mumbai, CreatedBy: System, CreatedOn: 25-02-2023 12:11:39
```

Null Substitution in Automapper:

The Null Substitution allows us to supply an alternate value for a destination member if the source value is null. That means instead of mapping the null value from the source object to the destination object, it will map the value we supply. We need to use the **NullSubstitute()** method to substitute the null value using AutoMapper. Let us understand how to use **NullSubstitute()** in AutoMapper with an example. We are going to work with the same PermanentAddress and TemporaryAddress classes.

So, what our requirement is, if the Address Property Value is null in the Source Object, then we need to store NA in the Address property in the destination object. For this, we need to use NullSubstitute() method. So, modify the MapperConfig class as follows to use NullSubstitute() method on the Address property.

```
using AutoMapper;
using System;
namespace AutoMapperDemo
{
    public class MapperConfig
    {
        public static Mapper InitializeAutomapper()
        {
            //Configuring AutoMapper
            var config = new MapperConfiguration(cfg =>
            {
                cfg.CreateMap<PermanentAddress, TemporaryAddress>()
                    //Using MapFrom Method to Store Static or Hard-Coded Value
                    //Destination Property
                    .ForMember(dest => dest.CreatedBy, act => act.MapFrom(src
                    "System"))

                    //Using MapFrom Method to Store Dynamic Value in a Destination
                    //Property
                    //Here, we are calling GetCurrentDateTime method which will
                    //dynamic value
                    .ForMember(dest => dest.CreatedOn, opt => opt.MapFrom(src =>
                    GetCurrentDateTime()))
                    //Storing NA in the destination Address Property, if Source
                    //is NULL
                    .ForMember(dest => dest.Address, act => act.NullSubstitute(
                    "NA").ReverseMap());
            });

            //Creating the Mapper Instance
            var mapper = new Mapper(config);
            //returning the Mapper Instance
            return mapper;
        }
    }
}
```

```

    }
    //Method to return Dynamic Value
    public static DateTime GetCurrentDateTime()
    {
        //Write the Logic to Get Dynamic value
        return DateTime.Now;
    }
}

```

Next, modify the Main method of the Program class as follows. Here, we are making the Address property value NULL.

```

using System;
namespace AutoMapperDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Initialize AutoMapper
            var mapper = MapperConfig.InitializeAutomapper();
            PermanentAddress permAddress = new PermanentAddress()
            {
                Name = "Pranaya",
                Address = null,
                CreatedBy = "Dot Net Tutorials",
                CreatedOn = DateTime.Now
            };
            var TempAddress = mapper.Map<PermanentAddress, TemporaryAddress>
(permAddress);
            Console.WriteLine("After Mapping Permanent Address");
            //Here CreatedBy and CreatedOn will be empty for Permanent Address
            Console.WriteLine($"Name: {permAddress.Name}, Address: {permAddress.Address},
CreatedBy: {permAddress.CreatedBy}, CreatedOn: {permAddress.CreatedOn}");
            Console.WriteLine("After Mapping Permanent Address");
            //Here CreatedBy with Fixed Value and CreatedOn with Dynamic Value
            Console.WriteLine($"Name: {TempAddress.Name}, Address: {TempAddress.Address},
CreatedBy: {TempAddress.CreatedBy}, CreatedOn: {TempAddress.CreatedOn}");
            Console.ReadLine();
        }
    }
}

```

When we run the application, it will give us the following output. Here, you can see, in the Source Object, it is printing the Address value as NULL (here it is not showing), but in the destination Address property, it is printing the value as NA.

```
After Mapping Permanent Address
```

```
Name: Pranaya, Address: , CreatedBy: Dot Net Tutorials, CreatedOn: 25-02-2023 12:25:13
```

```
After Mapping Permanent Address
```

```
Name: Pranaya, Address: N/A, CreatedBy: System, CreatedOn: 25-02-2023 12:25:13
```

In this article, I try to explain **How to Store Fixed and Dynamic Values in Destination Property in AutoMapper with Examples using C#**. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this How to Store Fixed and Dynamic Values in Destination Property in AutoMapper with Examples article.